

Lecture 2: Python Essentials for Backend Development

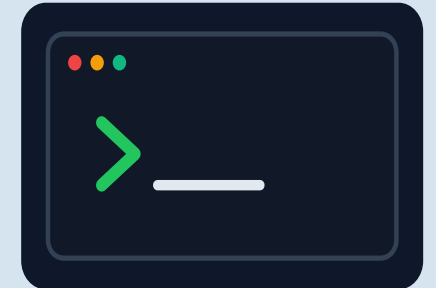
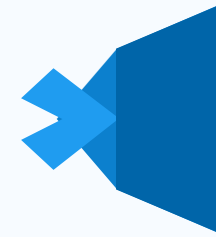
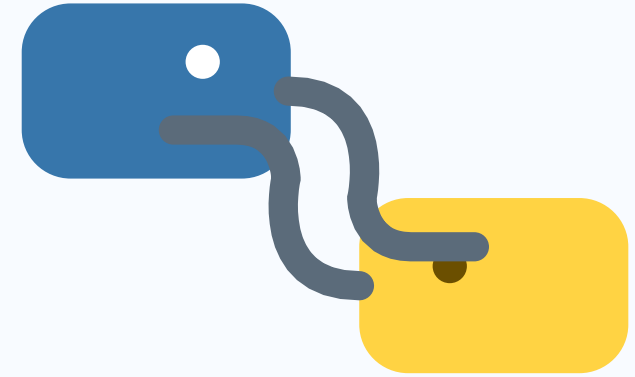
Second-year students | Detailed, example-driven, installation-aware lecture

Detailed lecture deck for in-class explanation, live coding, practice, and homework

What students should feel after this lecture

“I can write simple Python code by myself, run it correctly, read errors, and understand the building blocks needed before Django.”

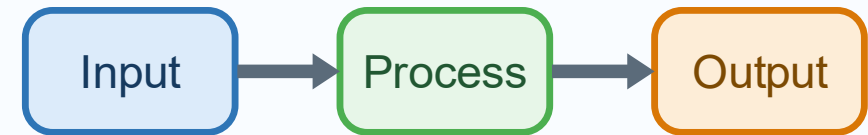
Dr. Mohamed Nour Eldien



Lecture Objectives

By the end of this lecture, students should be able to:

- Explain why Python is a practical language for backend development.
- Install and run the core tools: Python, VS Code, and the integrated terminal.
- Write simple Python programs using variables, data types, input, and output.
- Use decisions, loops, functions, collections, modules, and basic file handling.
- Understand how these basics will later appear inside Django projects.



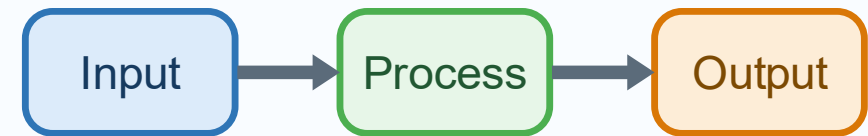
Suggested pacing

Use this as your opening map for the entire lecture. Tell students that Python syntax is only the start; problem solving is the real target.

Lecture Roadmap

A 60-minute explanation needs a visible structure

- Part 1: Tools and environment setup.
- Part 2: Core Python syntax and data types.
- Part 3: Conditions and loops.
- Part 4: Data structures and functions.
- Part 5: Modules, errors, files, and how all of this supports backend work.



Teaching Tip

Students feel safer when they know where the lecture is going. Keep referring back to this structure.

Part 1 - Tools and Environment

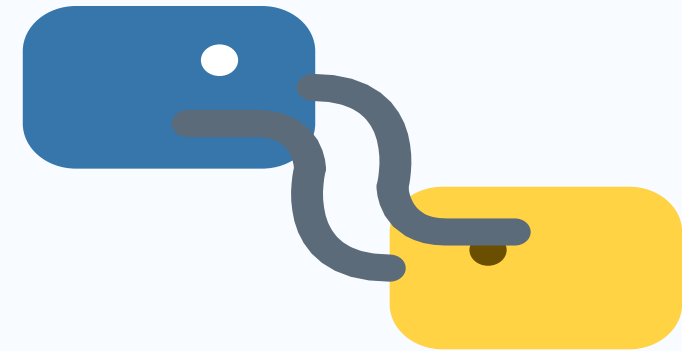
Students must know how to install, open, and run before they can code confidently.



Why Python for Backend?

Before syntax, explain the big reason

- Python is readable. Students can focus on logic instead of complex punctuation.
- Python is productive. We can build prototypes quickly and test ideas fast.
- Python has a large ecosystem. Django, Flask, FastAPI, Pandas, and many more libraries exist.
- Python is friendly for education. It helps students move from theory to practice smoothly.



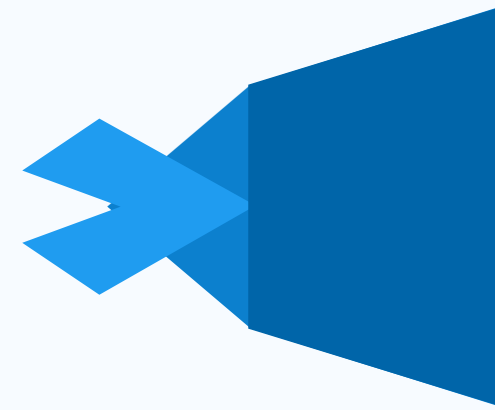
How to explain

Contrast Python with low-level languages: Python lets us express ideas with fewer lines, which is valuable in backend learning.

Programs We Need Today

Keep the toolchain simple in the beginning

- Python: the language interpreter that runs our code.
- VS Code: a lightweight editor to write, save, and organize code files.
- Python Extension for VS Code: gives syntax highlighting and useful coding support.
- Terminal / Command Prompt: runs programs and checks the environment.



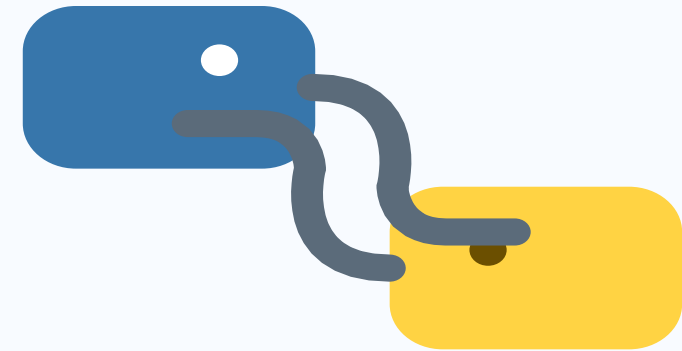
Classroom Tip

Tell students that professional programmers still use these same basic tools every day.

Step 1 - Download Python

Explain this as a real procedure students can repeat at home

- Open the official Python website in the browser.
- Choose the latest stable Python 3 release.
- During installation on Windows, check “Add Python to PATH”.
- Complete the installation and keep the default options if unsure.



Important warning

The “Add Python to PATH” checkbox is crucial. If students miss it, the terminal may not recognize python.

Verify the Python Installation

Do not assume the installation worked; always test it

- Open Command Prompt or PowerShell.
- Type one of these commands and press Enter.
- If Python is installed correctly, the version number appears.
- If the command is not recognized, PATH is probably not configured.

Live Code

```
1 python --version
2 py --version
```

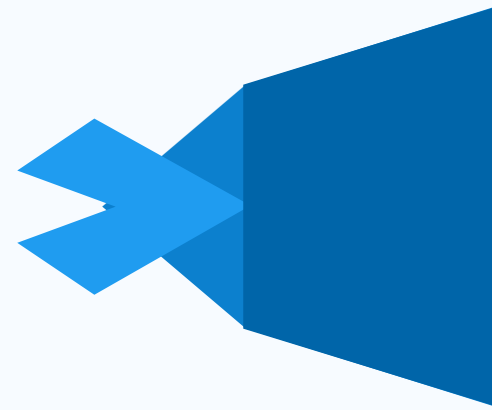
Teaching Tip

Demonstrate both commands. On many Windows machines, `py` works even when `python` does not.

Step 2 - Download VS Code

Students need an editor that is simple and modern

- Open the official VS Code website.
- Download the installer for Windows.
- Install it with the default settings.
- Open VS Code and notice the Explorer, Editor, and Terminal areas.



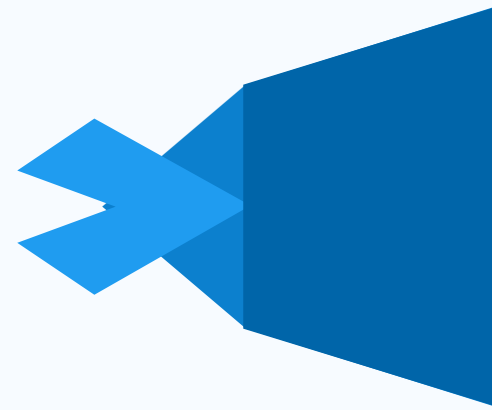
Explain visually

Point to the three most important areas: file explorer, code editor, and terminal panel.

Step 3 - Install the Python Extension

This is optional for running code, but important for a good learning experience

- Open VS Code.
- Click Extensions on the left sidebar.
- Search for “Python” published by Microsoft.
- Install it so code becomes easier to read, run, and debug.



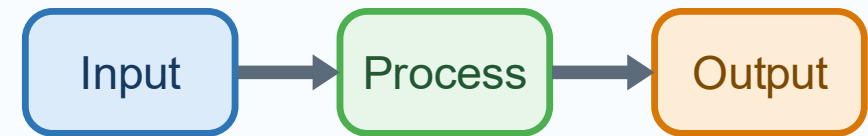
Why it matters

Without the extension, students can still write code, but the experience is less friendly and more error-prone.

Create a Working Folder

Teach students early how to stay organized

- Create a folder such as `python_backend_course`.
- Inside it, make separate files like `lecture2_examples.py`.
- Keep names meaningful and avoid random desktop files.
- Good organization reduces mistakes later in Django projects.



Bridge to future lectures

Explain that Django itself is a structured project, so students should begin learning organization now.

Running the First Python File in VS Code

Students must see the full cycle: write, save, run, read output

- Create a file named first_program.py.
- Type the program shown on the right.
- Save the file.
- Open the terminal and run the script.

Live Code

```
1 print("Hello, Backend Students!")
2 print("Python is working correctly.")
```

Teaching Tip

Run it in the terminal using: `python first_program.py`

The Terminal Is Your Friend

Students often fear the terminal because it looks different

- The terminal is simply a text-based way to talk to the computer.
- We use it to run Python files, install packages, and execute framework commands.
- Later, Django commands like runserver and migrate will also be executed here.
- So learning the terminal now saves confusion later.



Reassure students

The terminal is not dangerous when used carefully; it is a normal part of modern development.

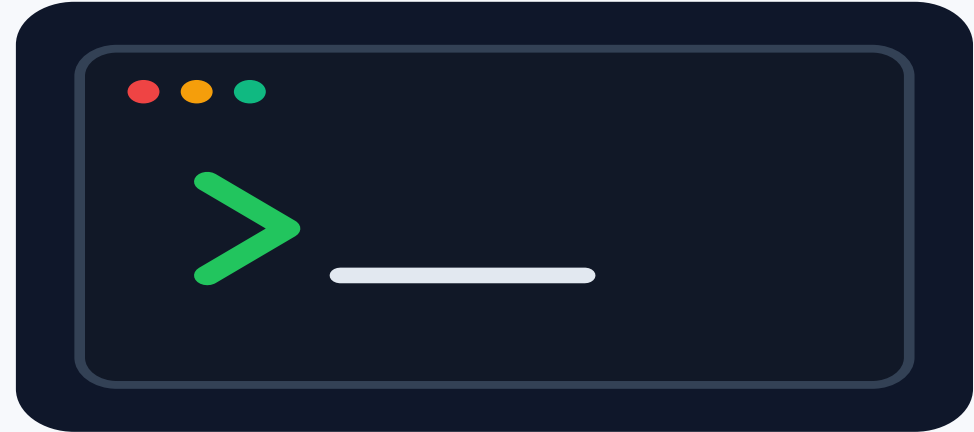
Quick Class Task - Tool Check

Practice is where programming starts to make sense

Class

Task

- Open VS Code.
- Create a folder for today's lecture.
- Create a file named test_run.py.
- Write one print statement and run it from the terminal.



Expected Direction

Walk around the room. Success means students can save and execute a file without help.

Part 2 - Python Basics

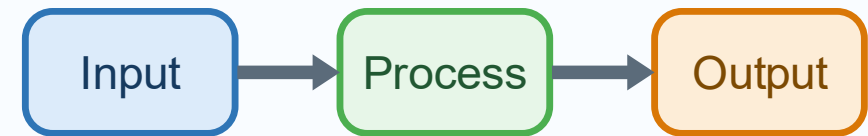
Now that the tools work, begin building the language itself.



What Is a Program?

A useful conceptual slide before syntax

- A program is a sequence of instructions given to a computer.
- Python reads these instructions from top to bottom.
- Each line should communicate a clear action.
- Good code is not just correct; it is also readable.



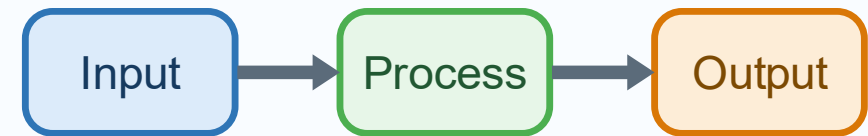
Explain with examples

Use simple examples like “print a message”, “store a value”, and “ask the user for input”.

Variables - Storing Information

A variable is a named place in memory

- Variables let us save data and use it later.
- A good variable name explains the meaning of the value.
- The assignment operator = stores the value inside the variable.
- Variables are fundamental because backend code constantly stores and manipulates data.



Live analogy

Use the “labeled box” analogy: the variable name is the label, the value is what is inside the box.

Variable Examples

Start with very small and readable examples

- Read each line aloud.
- Ask students what value each variable stores.
- Then show how print can display these values.
- Emphasize meaningful naming.

Live Code

```
1 student_name = "Mona"
2 age = 20
3 course = "Backend Python"
4 passed = True
5
6 print(student_name)
7 print(age)
8 print(course)
9 print(passed)
```

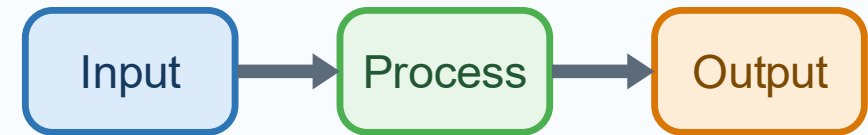
Teaching Tip

Ask: which names are clearer, student_name or x? This starts the style discussion early.

Rules for Naming Variables

Teach correctness and readability together

- A variable name can contain letters, digits, and underscore _.
- A variable name cannot start with a digit.
- Avoid spaces and reserved keywords like if, for, while.
- Use lowercase_with_underscores for readability in Python.



Important teaching move

Show both correct and incorrect examples: `user_name` is good, `2name` is not, `user name` is not.

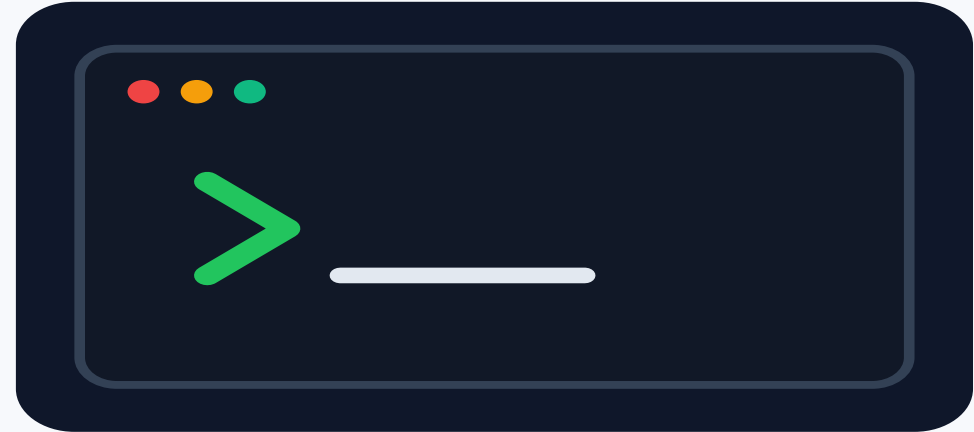
Practice - Fix the Variable Names

Practice is where programming starts to make sense

Class

Task

- Convert bad names into valid Python names.
- Example: student name -> student_name
- Example: 1score -> score_1
- Ask students to suggest better names for x, y, and z.



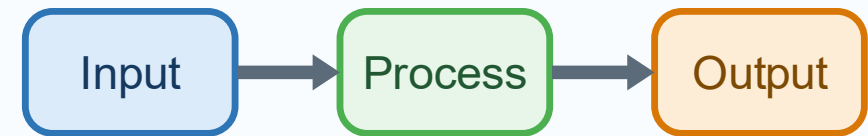
Expected Direction

The goal is not only syntax validity, but also good meaning. Encourage descriptive names.

Data Types - Why Type Matters

Backend systems care about what kind of data they are processing

- Different operations work on different data types.
- A string stores text, an integer stores whole numbers, and a float stores decimal numbers.
- A boolean stores True or False.
- Choosing the correct type reduces errors and makes code more logical.



Connect to backend

Explain that user names, emails, prices, and login states are all different kinds of data.

Strings - Text Data

Strings appear everywhere in web applications

- A string is text enclosed in single or double quotes.
- Strings are used for names, messages, IDs, addresses, and form data.
- We can combine strings using concatenation or f-strings.
- Readable output matters for users and debugging.

Live Code

```
1 name = "Ali"
2 city = "Cairo"
3
4 print("Student name: " + name)
5 print(f"{name} lives in {city}")
```

Teaching Tip

Show both concatenation and f-strings, but recommend f-strings for readability.

Integers and Floats

Numerical data is common in all applications

- Use int for whole numbers like age, count, year.
- Use float for decimal values like price or average.
- Python allows arithmetic directly on numeric types.
- The output changes if the type changes.

Live Code

```
1 age = 19
2 price = 149.75
3
4 print(age + 1)
5 print(price * 2)
```

Teaching Tip

Ask students: why should a phone number usually be stored as text, not as an integer?

Booleans - True or False

Booleans control program decisions

- A boolean has only two values: True and False.
- Booleans are used for conditions such as `logged_in`, `passed`, or `is_admin`.
- They often appear inside if statements.
- Backend logic depends heavily on boolean thinking.

Live Code

```
1 logged_in = True
2 is_admin = False
3
4 print(logged_in)
5 print(is_admin)
```

Teaching Tip

Bridge ahead: later, if statements will ask questions based on these values.

Using type()

Students should learn to inspect values when confused

- type() tells us the type of a variable or value.
- This helps when debugging input and conversion issues.
- Students should use it when results look strange.
- It is a simple but powerful habit.

Live Code

```
1 name = "Sara"
2 score = 95
3 average = 87.5
4 active = True
5
6 print(type(name))
7 print(type(score))
8 print(type(average))
9 print(type(active))
```

Teaching Tip

Mention that many beginner errors come from not realizing the actual type of a value.

Input and Output

Programming becomes interactive when the user gives data

- `print()` sends output to the screen.
- `input()` reads text entered by the user.
- `input()` returns a string, even if the user types a number.
- This is why type conversion often becomes necessary.

Live Code

```
1 name = input("Enter your name: ")
2 print("Welcome", name)
```

Teaching Tip

Very important: tell students that `input()` returns text. This will matter in later conversion examples.

Converting Input to int and float

Without conversion, arithmetic may behave incorrectly

- Use `int()` when the input should become a whole number.
- Use `float()` when decimals are allowed.
- Always think about what the real-world meaning of the data is.
- Type conversion is a bridge between user text and program logic.

Live Code

```
1 age = int(input("Enter your age: "))
2 height = float(input("Enter your height in meters: "))
3
4 print(age + 1)
5 print(height)
```

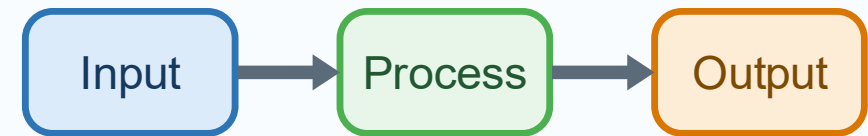
Teaching Tip

Ask: what happens if the user enters text instead of a number? Use this as a lead-in to errors later.

Comments in Python

Explain why comments exist, but warn against useless comments

- Comments start with the symbol #.
- They are ignored by Python during execution.
- Use comments to explain intent, not obvious code.
- Good comments help humans; good names reduce the need for too many comments.



Give one good and one bad comment

Bad: # print the name above print(name). Good: # Convert age to int because input returns text.

Part 3 - Operators, Conditions, and Loops

Now students begin controlling program behavior.



Arithmetic Operators

These are the building blocks of calculation

- Addition +, subtraction -, multiplication *, division /.
- Integer division // removes the fractional part.
- Remainder % is useful for even/odd checks.
- Exponent ** raises a number to a power.

Live Code

```
1 a = 10
2 b = 3
3
4 print(a + b)
5 print(a - b)
6 print(a * b)
7 print(a / b)
8 print(a // b)
9 print(a % b)
10 print(a ** b)
```

Teaching Tip

Spend extra time on // and % because students usually ask about them.

Comparison Operators

Programs make decisions by comparing values

- == checks equality, != checks inequality.
- > and < compare larger and smaller values.
- >= and <= include equality as well.
- The result of a comparison is always True or False.

Live Code

```
1 score = 75
2 print(score > 50)
3 print(score == 75)
4 print(score != 100)
```

Teaching Tip

Point out the difference between = for assignment and == for comparison.

Logical Operators

Use logic to combine more than one condition

- and means both conditions must be true.
- or means at least one condition is true.
- not reverses the boolean result.
- These operators are essential in backend validation.

Live Code

```
1 age = 20
2 has_id = True
3
4 print(age >= 18 and has_id)
5 print(age < 18 or has_id)
6 print(not has_id)
```

Teaching Tip

Use a real example: a system allows access if age >= 18 and the student has an ID.

if Statement

A decision starts with asking a question

- if runs a block of code only when the condition is True.
- Indentation is part of Python syntax, not only style.
- The colon : starts the block.
- This is one of the most important Python structures.

Live Code

```
1 score = 80
2
3 if score >= 50:
4     print("Passed")
```

Teaching Tip

Show how removing indentation causes a syntax or logic problem.

if - else

Programs often need two possible paths

- if handles the True case.
- else handles the False case.
- Together they express clear binary logic.
- This is common in login checks, validations, and status messages.

Live Code

```
1 score = 42
2
3 if score >= 50:
4     print("Passed")
5 else:
6     print("Failed")
```

Teaching Tip

Ask students to predict the output before running it.

if - elif - else

Use elif when there are multiple categories

- elif means “else if”.
- Python checks conditions from top to bottom.
- The first matching condition runs.
- This pattern is useful for grading, status labels, and classification.

Live Code

```
1 mark = 87
2
3 if mark >= 90:
4     print("A")
5 elif mark >= 80:
6     print("B")
7 elif mark >= 70:
8     print("C")
9 else:
10    print("D")
```

Teaching Tip

Stress order: higher thresholds should come before lower thresholds.

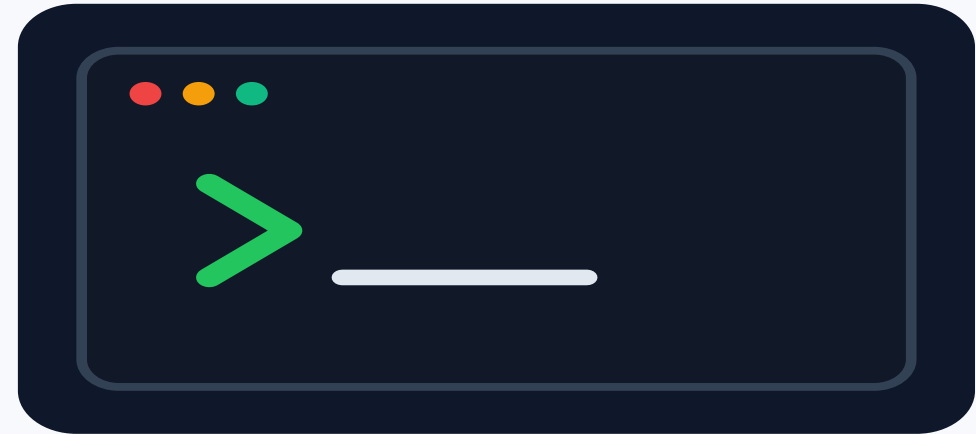
Practice - Write a Grade Classifier

Practice is where programming starts to make sense

Class

Task

- Ask the user to enter a numeric grade.
- Display Excellent if grade ≥ 85 .
- Display Good if grade ≥ 65 and < 85 .
- Display Need Improvement otherwise.



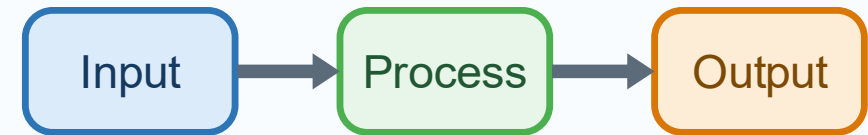
Expected Direction

Expected solution uses `input()`, `int()`, and `if-elif-else`.
Encourage students to test multiple values.

Nested Conditions

Sometimes one decision contains another decision

- A nested condition means an if statement inside another if statement.
- Use nesting only when it truly improves clarity.
- Too much nesting can make code hard to read.
- Backend code often nests checks for login, role, and permissions.



Classroom example

If user is logged in, then check role. If role is admin, then show admin panel.

Nested Condition Example

Use a realistic scenario students can imagine

- First check whether the user is logged in.
- Then check whether the user is an admin.
- Print a different message for each case.
- This mirrors backend authorization logic.

Live Code

```
1 logged_in = True
2 is_admin = False
3
4 if logged_in:
5     if is_admin:
6         print("Open admin dashboard")
7     else:
8         print("Open student dashboard")
9 else:
10    print("Please log in")
```

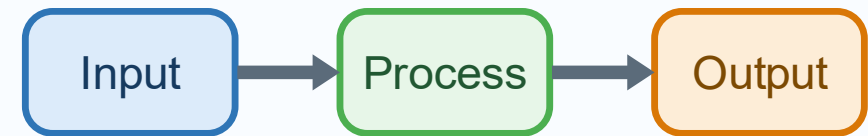
Teaching Tip

Bridge to web apps: in a real system, different users see different pages based on such checks.

Why Loops Matter

Loops save us from repeating code manually

- A loop repeats a block of code.
- Use loops when the same action must happen many times.
- Loops are essential for processing lists of records, names, files, or requests.
- In backend work, loops often handle collections of objects from a database.



Everyday analogy

Instead of printing ten names with ten print statements, a loop can handle them automatically.

while Loop

A while loop repeats while a condition remains True

- Initialize a variable before the loop.
- Update that variable inside the loop.
- If the condition never becomes False, the loop may never stop.
- So students must always think about the stopping condition.

Live Code

```
1 count = 1
2
3 while count <= 5:
4     print(count)
5     count += 1
```

Teaching Tip

Slowly trace the value of count on the board after each repetition.

for Loop with range()

for is often the preferred loop in Python

- for reads more naturally when the number of repetitions is known.
- range(5) means 0, 1, 2, 3, 4.
- range(start, stop) begins from start and stops before stop.
- This form is widely used in counting and indexing.

Live Code

```
1 for i in range(5):  
2     print(i)  
3  
4 for i in range(1, 6):  
5     print(i)
```

Teaching Tip

Spend time explaining that the stop value is excluded. This confuses many beginners.

Looping Through a String and a List

Loops become more powerful when they iterate over collections

- A for loop can visit each character in a string.
- It can also visit each item in a list.
- This is very natural Python code.
- Students should learn both patterns.

Live Code

```
1 name = "Omar"
2 for ch in name:
3     print(ch)
4
5 students = ["Ali", "Mona", "Sara"]
6 for student in students:
7     print(student)
```

Teaching Tip

Use this to prepare students for collections in the next part.

break, continue, and pass

These keywords modify loop behavior

- `break` exits the loop immediately.
- `continue` skips the current iteration and moves to the next one.
- `pass` does nothing; it is a placeholder.
- Use them carefully because they can also reduce clarity if overused.

Live Code

```
1 for i in range(1, 6):  
2     if i == 3:  
3         continue  
4     if i == 5:  
5         break  
6     print(i)
```

Teaching Tip

Trace the output with students and ask which values are printed and why.

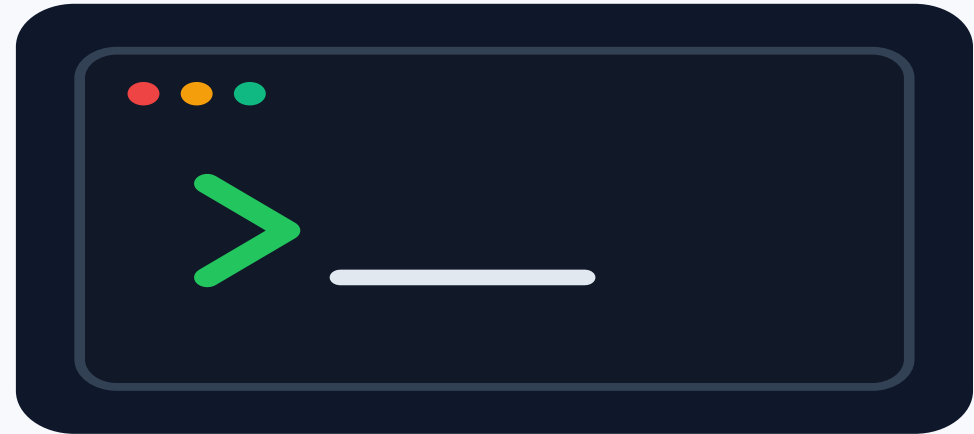
Practice - Multiplication Table

Practice is where programming starts to make sense

Class

Task

- Ask the user for a number.
- Use a for loop to print the multiplication table from 1 to 10.
- Format the output clearly.
- Optional: ask students to add a title line before the table.



Expected Direction

Expected solution uses `input()`, `int()`, `range(1, 11)`, and print formatting.

Part 4 - Data Structures and Functions

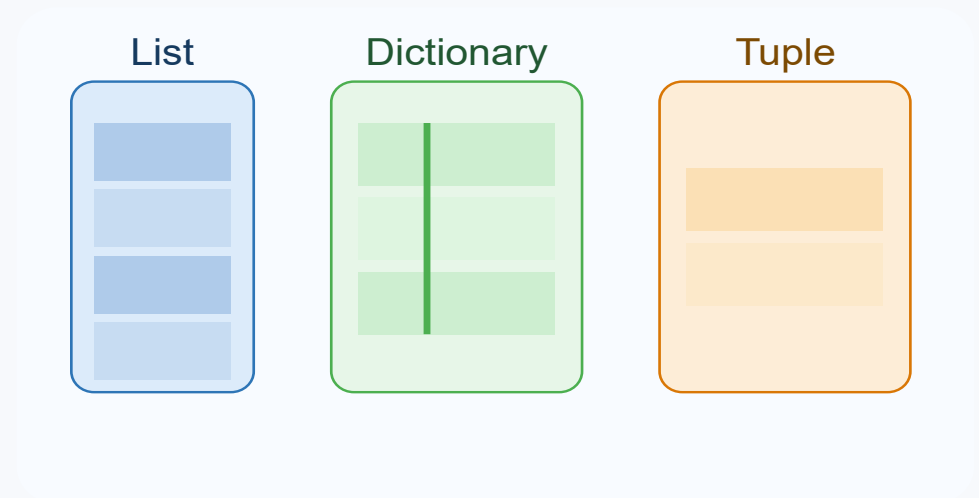
This part moves from single values to organized collections and reusable logic.



Why Data Structures Matter

One value is rarely enough in real applications

- A backend system usually handles many pieces of information together.
- We may need a list of students, a dictionary of profile data, or a set of unique values.
- Data structures help us organize information correctly.
- The correct structure makes later code cleaner and faster to understand.



Relate to web forms

A submitted form often contains multiple fields. We need organized ways to store them.

Lists

A list stores multiple items in order

- Lists use square brackets [].
- Items keep their order and can be changed.
- Lists are very common for names, products, tasks, and results.
- Each item has an index starting from 0.

Live Code

```
1 students = ["Ali", "Mona", "Sara"]
2 print(students)
3 print(students[0])
4 print(students[2])
```

Teaching Tip

Explain indexing carefully. Students must understand that the first item is index 0.

Common List Operations

Students should learn to modify lists, not only read them

- `append()` adds one item at the end.
- `remove()` deletes an item by value.
- `len()` gives the number of items.
- Lists work very naturally with loops.

Live Code

```
1 students = ["Ali", "Mona"]
2 students.append("Sara")
3 students.remove("Ali")
4
5 print(students)
6 print(len(students))
```

Teaching Tip

Show the state of the list after each operation.

Tuples

A tuple is like a list but fixed

- Tuples use parentheses ().
- They preserve order but cannot be modified after creation.
- Use tuples when the data should stay constant.
- Examples include coordinates, fixed settings, or known pairs.

Live Code

```
1 point = (10, 20)
2 print(point)
3 print(point[0])
```

Teaching Tip

Keep it simple: the main idea is that tuples are ordered but immutable.

Sets

Sets store unique values

- Sets use curly braces {} or set().
- A set does not keep duplicate values.
- Use sets when uniqueness matters.
- They are useful for tags, unique IDs, or removing repeated entries.

Live Code

```
1 colors = {"red", "blue", "red", "green"}  
2 print(colors)
```

Teaching Tip

Explain that the repeated value red appears only once.

Dictionaries

A dictionary stores data as key-value pairs

- Use a dictionary when values have labels.
- This is excellent for user profiles, product data, or settings.
- Keys should be meaningful and usually unique.
- Dictionaries are a very backend-friendly structure.

Live Code

```
1 student = {  
2     "name": "Laila",  
3     "age": 20,  
4     "department": "IS"  
5 }  
6  
7 print(student["name"])  
8 print(student["department"])
```

Teaching Tip

Relate this directly to JSON and web data, but keep the explanation simple.

Updating a Dictionary

A dictionary can be read and modified easily

- Use a key to access a value.
- Assign a new value to update existing data.
- Add a new key-value pair by assigning a new key.
- This pattern feels similar to updating records.

Live Code

```
1 student = {"name": "Laila", "age": 20}
2 student["age"] = 21
3 student["department"] = "CS"
4
5 print(student)
```

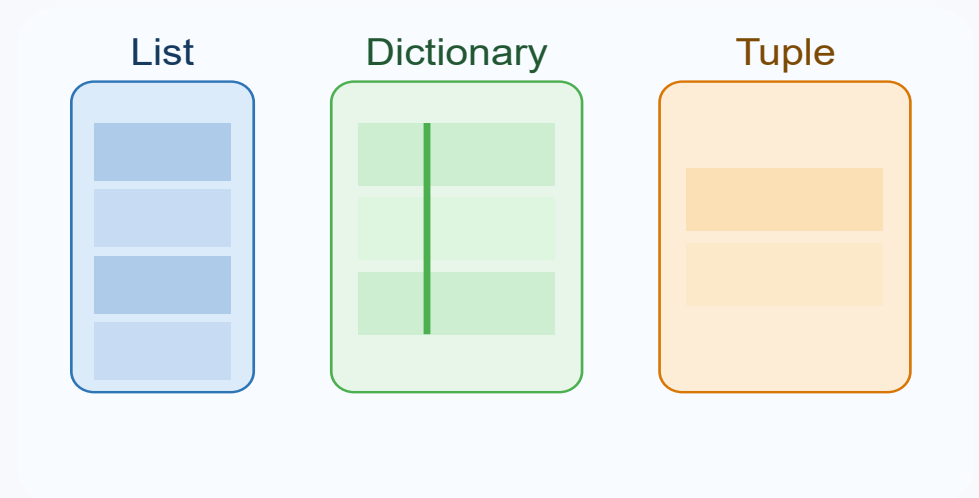
Teaching Tip

Show before and after to highlight mutation.

How to Choose the Right Structure

Students often ask which structure to use

- Use list when order matters and items may change.
- Use tuple when order matters but items should stay fixed.
- Use set when uniqueness matters more than order.
- Use dictionary when each value has a clear label or key.



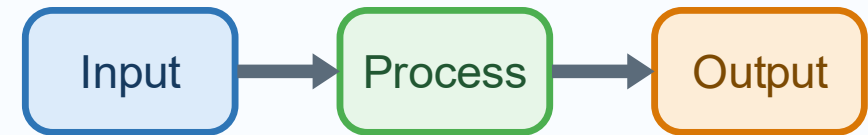
Ask a question

For student data with name, age, and department, which structure is best? The answer should be dictionary.

Why Functions Matter

Functions make code reusable and cleaner

- A function groups code that performs one task.
- Instead of repeating the same code many times, define it once and call it when needed.
- Functions improve readability, testing, and maintenance.
- Backend frameworks depend heavily on functions and reusable blocks.



Bridge to Django views

Tell students that later a Django view is also a function that receives a request and returns a response.

Defining and Calling a Function

Start with the simplest possible form

- Use `def` to define a function.
- The function name should describe its action.
- Call the function by writing its name with parentheses.
- Indent the function body correctly.

Live Code

```
1 def welcome():  
2     print("Welcome to Backend Python")  
3  
4 welcome()
```

Teaching Tip

Ask students to explain the difference between defining a function and calling it.

Function Parameters

Parameters let one function work with many values

- A parameter is a variable inside the function definition.
- An argument is the actual value passed during the call.
- This makes the function flexible and reusable.
- Backend systems constantly pass data into functions.

Live Code

```
1 def greet(name):  
2     print(f"Hello, {name}")  
3  
4 greet("Mona")  
5 greet("Omar")
```

Teaching Tip

Use the terms parameter and argument carefully; repeat them more than once.

return Statement

A function can send a result back to the caller

- print shows output immediately, but return gives a value back.
- Returned values can be stored in variables or used later.
- This is more powerful than always printing directly.
- Most backend functions return values that are processed by other parts of the program.

Live Code

```
1 def add(a, b):  
2     return a + b  
3  
4 result = add(5, 7)  
5 print(result)
```

Teaching Tip

Explain the difference clearly: print displays, return hands back a value.

Local Variables and Scope

Variables do not all live in the same place

- A variable created inside a function is usually local to that function.
- It cannot normally be used outside the function.
- This protects code from accidental interference.
- Understanding scope prevents many confusing errors.

Live Code

```
1 def test_scope():
2     message = "Inside function"
3     print(message)
4
5 test_scope()
6 # print(message) # This would cause an error
```

Teaching Tip

Explain that scope is one reason functions help organize larger programs.

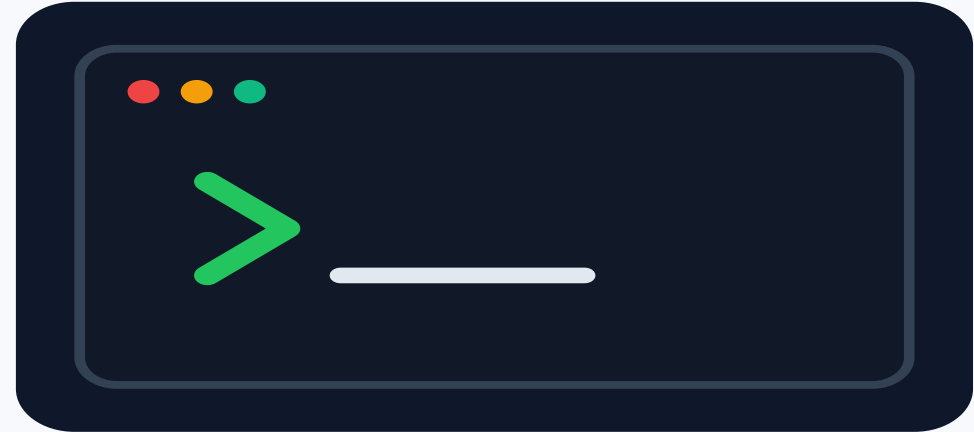
Practice - Build Three Functions

Practice is where programming starts to make sense

Class

Task

- Write a function that returns the square of a number.
- Write a function that checks whether a number is even.
- Write a function that receives a student name and prints a welcome message.
- Test each function with at least two examples.

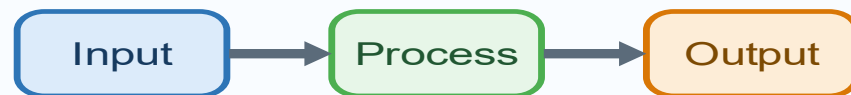


Expected Direction

Encourage students to create one file and solve all three tasks inside it.

Part 5 - Modules, Errors, Files, and Backend Connection

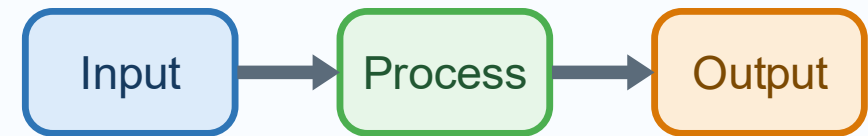
Students now see how Python grows beyond a single script.



What Is a Module?

A module is simply a Python file that contains reusable code

- Modules help us split programs into logical parts.
- We can import built-in modules like math.
- We can also create our own modules.
- Django itself is a collection of many modules working together.



Make the future connection

When students later import from `django.shortcuts` or `django.urls`, they are importing from modules.

Importing a Built-in Module

Use a tiny example that students can run immediately

- Import the math module.
- Use `math.sqrt()` or `math.pi`.
- Show that Python already comes with many useful modules.
- Importing keeps code organized and avoids rewriting everything.

Live Code

```
1 import math
2
3 print(math.sqrt(25))
4 print(math.pi)
```

Teaching Tip

Mention that many powerful tools come from importing libraries, not writing everything manually.

Creating Your Own Module

This is a gentle introduction to code organization

- Create one file named helpers.py.
- Define a function inside it.
- Create another file and import that function.
- This shows students how code can be split across files.

Live Code

```
1 # helpers.py
2 def greet(name):
3     return f"Hello, {name}"
4
5 # main.py
6 from helpers import greet
7 print(greet("Nour"))
```

Teaching Tip

Explain that Python looks for the other file in the same folder.

Errors Are Part of Programming

Normalize mistakes early so students do not panic

- An error message is not a sign of failure; it is information.
- Good programmers read errors carefully instead of guessing blindly.
- The line number and error type are usually the first clues.
- Debugging is a normal skill, not a punishment.



Encouragement

Tell students clearly: every programmer sees errors daily, even professionals.

try and except

Basic error handling should be introduced gently

- Some code may fail during execution.
- try lets us test risky code.
- except lets us handle the error instead of crashing badly.
- This is useful when dealing with user input and file operations.

Live Code

```
1 try:
2     num = int(input("Enter a number: "))
3     print(num * 2)
4 except:
5     print("Invalid input")
```

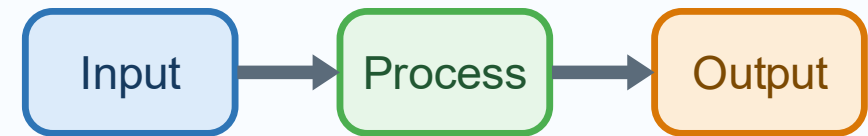
Teaching Tip

Keep this basic. Mention later they will learn more specific exceptions.

Common Beginner Errors

Prepare students to diagnose frequent problems

- `SyntaxError`: Python cannot understand the written code structure.
- `NameError`: a variable is used before it is defined.
- `TypeError`: an operation is used on the wrong type.
- `IndentationError`: spacing is wrong in a language where indentation matters.



Good teaching move

Show one tiny example of each type verbally, even if not all are coded on the slide.

Debugging Tips

Give students a method, not only motivation

- Read the error message slowly.
- Check the line number shown in the terminal.
- Print intermediate values to inspect them.
- Simplify the program until the problem becomes easier to isolate.



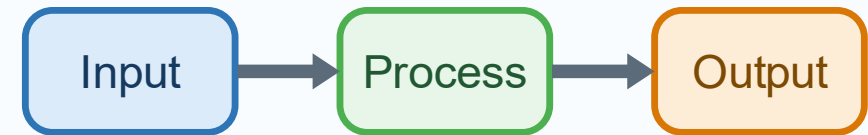
Classroom script

Say: do not change ten things at once. Change one thing, test, observe, and continue.

Files in Python

Real applications often need to read or write data

- A file is external stored data on disk.
- Programs can read existing files or create new ones.
- This is useful for logs, reports, simple data storage, and configuration.
- In backend systems, file handling is common for uploads, exports, and text processing.



Bridge to real systems

Mention uploaded files, reports, and logs as realistic backend examples.

Reading from a File

Use a small and readable example

- Create a text file named notes.txt in the same folder.
- Use open() with mode "r" for reading.
- Read the contents and print them.
- Close the file or use with later in advanced examples.

Live Code

```
1 file = open("notes.txt", "r")
2 content = file.read()
3 print(content)
4 file.close()
```

Teaching Tip

Do not go too deep. The goal is simply to show the concept of external data.

Writing to a File

Students should see that programs can also produce output files

- Use mode "w" to write into a file.
- If the file does not exist, Python creates it.
- If it exists, writing may replace old content.
- Always explain this risk so students are careful.

Live Code

```
1 file = open("report.txt", "w")
2 file.write("Backend Python Lecture 2")
3 file.close()
```

Teaching Tip

Stress that mode w can overwrite content. This is a practical warning.

Mini Program - Student Information

End the technical part with a small complete example

- Take user input for name and score.
- Convert score to int.
- Use a condition to decide pass/fail.
- Print a final message.

Live Code

```
1 name = input("Enter student name: ")
2 score = int(input("Enter score: "))
3
4 if score >= 50:
5     status = "Passed"
6 else:
7     status = "Failed"
8
9 print(f"{name} -> {status}")
```

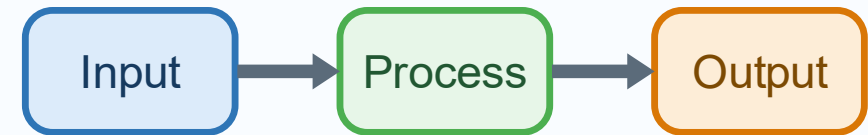
Teaching Tip

This example combines input, conversion, condition, and output into one coherent mini application.

Python Style Habits to Start Today

Style affects readability and teamwork

- Use meaningful names.
- Indent consistently with 4 spaces.
- Do not write extremely long and confusing lines.
- Leave blank lines between logical blocks of code when it improves readability.



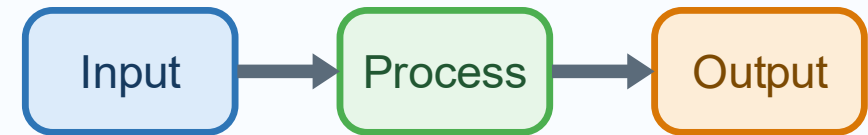
State the principle

Clean code is a gift to your future self and to anyone who reads your program.

How Today's Topics Connect to Django

Students should see the future purpose of every concept

- Variables and data types appear in form handling and program logic.
- Conditions appear in validation and access control.
- Loops appear when processing query results and repeated content.
- Functions appear in views and helper utilities.
- Modules appear in project organization.



Very important bridge

This slide makes the lecture feel purposeful. Without this bridge, Python may seem disconnected from backend development.

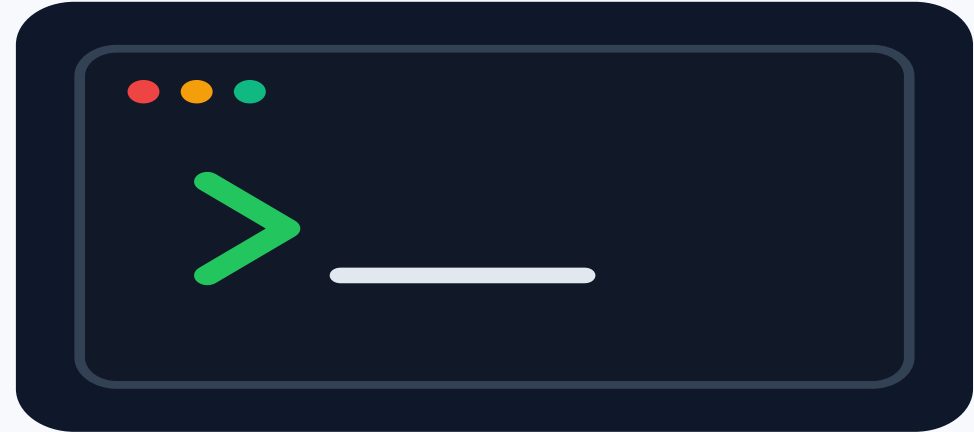
Final In-Class Activity

Practice is where programming starts to make sense

Class

Task

- Build a small script that asks for a student name, age, and three quiz marks.
- Store the marks in a list.
- Calculate the average.
- Print whether the student passed or failed based on the average.



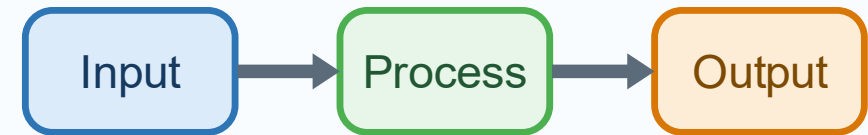
Expected Direction

Students may solve this in multiple ways. Accept different correct solutions, but ask them to explain their logic.

Mini Quiz Questions

Use these orally or on the board at the end

- What is the difference between `=` and `==` ?
- Why does `input()` often require `int()` or `float()` afterward?
- When should we use a dictionary instead of a list?
- What is the difference between `print` and `return`?
- Why is indentation important in Python?



Use this interactively

Take answers from several students instead of answering everything yourself.

Homework

Homework should reinforce execution, not memorization only

- Write a Python program that stores five course names in a list and prints them one by one.
- Write a function that receives a mark and returns Pass or Fail.
- Create a dictionary for one student with name, level, and department.
- Optional challenge: save one result message into a text file.



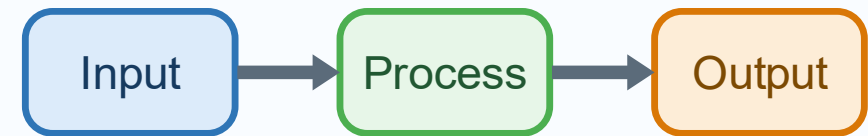
Homework tip

Ask students to submit both the code and a screenshot of successful execution.

Preview of Lecture 3

Keep motivation high for the next class

- Lecture 3 will move into Object-Oriented Programming (OOP).
- Students will learn classes, objects, attributes, methods, and constructors.
- This will prepare them for Django models and more structured code.
- So today's basics are the foundation for tomorrow's organization.



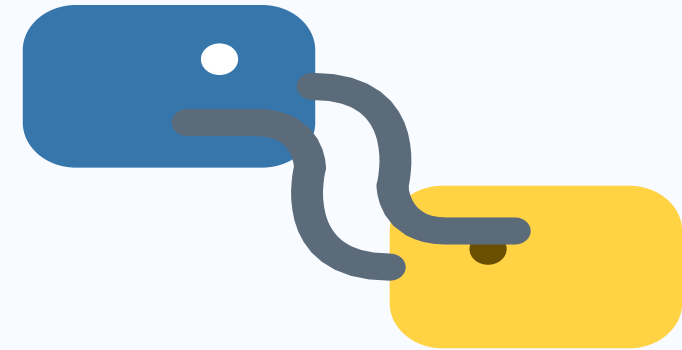
Closing message

End with confidence: today students learned how to think and work in Python, not only how to type code.

Thank You - Q&A

Invite questions about code, tools, or concepts

- Ask students what still feels confusing.
- Re-run any small code example if needed.
- Remind them to practice at home on the same machine.
- Encourage them not to fear errors during practice.



Last classroom move

Stay for questions about installation problems; these small issues can block learning in future lectures.